

CSI370 Computer Architecture

Software Project: Verte Compiler - Assembly Refactor

Andy Zheng

Champlain College

`andy.zheng@mymail.champlain.edu`

November 4, 2025

1 Project Overview

1.1 What: Project Description

This project will involve refactoring the code generation backend of the `Verte` compiler. `Verte` is an existing compiler project written in C++ that currently uses `LLVM` for code generation. The objective of this project is to replace the `LLVM` backend with a custom, hand-written `x86-64 Assembly` code generator. This new backend will directly translate the `AST` produced by the `Verte` frontend into `x86-64 Assembly` instructions.

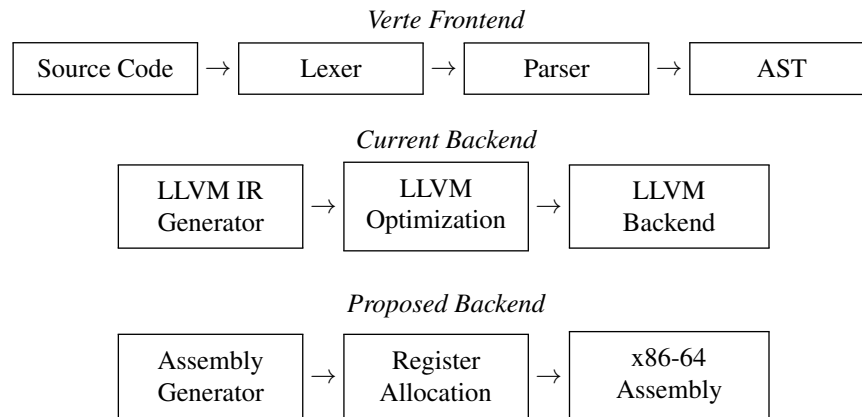


Figure 1: High-level view of the `Verte` Compiler architecture, showing existing and proposed backends.

1.2 Why: Motivation

The motivation behind this project is for both educational and technical reasons. I believe that building a custom assembly backend will provide invaluable insights into low-level code generation, and compilation. And, specifically in the context of this class, I will also aim to deepen my understanding of `x86-64 Assembly` instructions, calling conventions, and register/instruction selection. As for the technical reasons, I feel it would be a good idea to explore alternatives to `LLVM` due to its complexity and steep learning curve. Thus, overall a custom backend will allow for more understanding and control over the generated code.

1.3 How: Approach and Methodology

The implementation of this refactor will be carried out in several phases over the rest of this course. It will start with an analysis of the existing `LLVM` backend integration to understand best how to replace it. Following that, designing the architecture for the assembly generation will be done. Whilst development is ongoing, test cases will be created to validate correctness, once assembly generation is functional and correctness is ensured, the refactorization is complete.

File	Purpose	Implementation
asm_visitor.h/cpp	AST traversal	Visitor pattern for AST nodes
instruction_selector.h/cpp	Instruction mapping	x86-64 instruction selection
register_allocator.h/cpp	Register management	Physical register allocation
asm_emitter.h/cpp	Code emission	NASM assembly text generation
calling_convention.h/cpp	ABI compliance	System V calling convention
backend.h/cpp	Integration	Replace LLVM backend interface

Table 1: Key source files and their roles in the assembly backend implementation

1.4 Challenges: Anticipated Difficulties

The project presents several technical and integration challenges that will need to be addressed. Specifically, managing register allocation and stack frames in x86-64 Assembly can be complex. Proper handling of function calls and adhering to calling conventions will also be critical to ensure generated code correctness. Thus careful planning and testing will be essential to overcome these challenges.

1.4.1 Solutions

To mitigate the anticipated challenges, I plan to take a systematic approach. This includes thorough research into x86-64 Assembly programming and calling conventions. Additionally, I will implement the backend incrementally, starting with simple features and gradually adding complexity. Testing will be conducted at each stage to ensure correctness and to catch issues early. Finally, I will leverage existing resources and tools, such as assemblers and debuggers, to aid in development and troubleshooting.

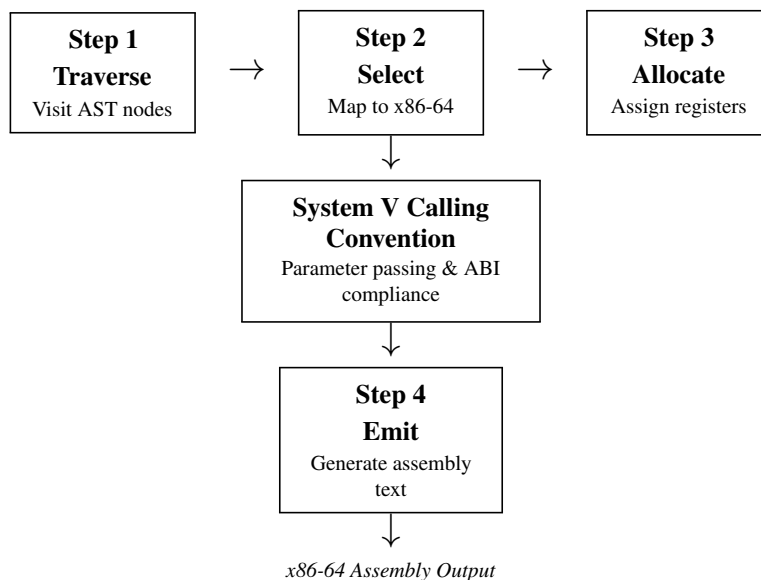


Figure 2: Incremental workflow from transforming AST into x86-64 Assembly code with System V ABI compliance.